

Microsoft SQL Server: Tipi di Dati, Operatori e Schema ScuolaDB

Docente: **Moussa Bakabi Salisou**

In questa dispensa approfondiamo le fondamenta di Microsoft SQL Server (T-SQL) utilizzando un caso di studio reale: il database **ScuolaDB**. Vengono esaminati in dettaglio i tipi di dati nativi, gli operatori di calcolo, di confronto e logici necessari per manipolare e interrogare le tabelle con precisione e performance.

Modulo 1: I Tipi di Dati in SQL Server

Scegliere il tipo di dato corretto per ogni colonna è essenziale per ottimizzare lo spazio su disco e garantire l'integrità dei dati. Ecco una panoramica delle categorie principali disponibili in T-SQL:

1.1 Tipi Numerici

Tipo di Dato	Spazio Occupato	Descrizione e Utilizzo Tipico
INT	4 Byte	Numeri interi standard (da circa -2 miliardi a +2 miliardi). Usato per ID autoincrementali.
BIGINT	8 Byte	Numeri interi molto grandi, ideali per sistemi ad alto traffico transazionale.
SMALLINT	2 Byte	Numeri interi ridotti (da -32.768 a 32.767). Ottimo per piccoli intervalli.
TINYINT	1 Byte	Numeri interi da 0 a 255. Perfetto per i voti scolastici o gli stati logici.
DECIMAL(p, s)	Variabile	Numeri decimali esatti con precisione p (totale cifre) e scala s (cifre decimali). Ideale per prezzi o medie pesate.

1.2 Tipi Stringa (Testuali)

Tipo di Dato	Supporto Unicode	Descrizione e Utilizzo Tipico
VARCHAR(n)	No (1 byte/carattere)	Stringhe di lunghezza variabile fino a n caratteri. Ideale per Email, Cognomi.
NVARCHAR(n)	Sì (2 byte/carattere)	Stringhe a lunghezza variabile internazionali. Obbligatorio per supportare caratteri speciali o alfabeti non occidentali.
CHAR(n)	No	Stringa a lunghezza fissa. Se il testo inserito è più corto, viene riempito con spazi vuoti. Ottimo per Codici Stato (es. 'IT', 'FR').

1.3 Tipi Data e Ora

- **DATE** : Memorizza solo la data (AAAA-MM-GG). Ottimo per le date di nascita o iscrizione.
- **TIME** : Memorizza solo l'orario (HH:MM:SS.nnnnnnnn).
- **DATETIME2** : Memorizza combinazione di data e ora con un'altissima precisione sui millisecondi. Da preferire al vecchio **DATETIME** .

Modulo 2: Gli Operatori in T-SQL

Gli operatori permettono di manipolare i valori, effettuare confronti all'interno delle clausole di filtro e costruire espressioni matematiche complesse.

2.1 Operatori Aritmetici

Utilizzati per i calcoli numerici: **+** (Addizione), **-** (Sottrazione), ***** (Moltiplicazione), **/** (Divisione), e **%** (Modulo / Resto della divisione).

2.2 Operatori di Confronto

Restituiscono un valore booleano (**TRUE** , **FALSE** o **UNKNOWN** se sono coinvolti dei valori **NULL**):

Operatore	Significato	Esempio d'Uso
=	Uguale a	<code>WHERE ClasseID = 3</code>
<> oppure !=	Diverso da	<code>WHERE Stato <> 'Sospeso'</code>
> / <	Maggiore di / Minore di	<code>WHERE Voto > 6.5</code>
>= / <=	Maggiore o Uguale / Minore o Uguale	<code>WHERE DataNascita >= '2010-01-01'</code>

2.3 Operatori Logici

Servono per combinare più filtri o invertire il risultato di una condizione:

- **AND** : La condizione totale è vera solo se **tutte** le sotto-condizioni sono vere.
- **OR** : La condizione totale è vera se **almeno una** delle sotto-condizioni è vera.
- **NOT** : Inverte il valore logico della condizione (es. `NOT IN` , `NOT LIKE`).

Modulo 3: Implementazione Pratica del Database "ScuolaDB"

Mettiamo in pratica i concetti studiati progettando la struttura di **ScuolaDB**. Lo schema prevede la gestione di Classi, Studenti e Voti d'esame.

3.1 Script DDL di Creazione dello Schema

```
-- Crea e seleziona il nuovo database richiesto
CREATE DATABASE ScuolaDB;
GO
USE ScuolaDB;
GO

-- 1. Tabella Classi
CREATE TABLE Classi (
    ClasseID INT IDENTITY(1,1) PRIMARY KEY,
    Sezione CHAR(2) NOT NULL, -- Esempio: '1A', '3B'
    AnnoScolastico VARCHAR(9) NOT NULL -- Esempio: '2025/2026'
);

-- 2. Tabella Studenti (Uso di VARCHAR, INT e DATE)
CREATE TABLE Studenti (
    StudenteID INT IDENTITY(1,1) PRIMARY KEY,
    Nome VARCHAR(50) NOT NULL,
    Cognome VARCHAR(50) NOT NULL,
    DataNascita DATE NOT NULL,
    Email VARCHAR(100) UNIQUE,
    ClasseID INT FOREIGN KEY REFERENCES Classi(ClasseID)
);

-- 3. Tabella Voti (Uso di TINYINT per i voti e DATETIME2 per la tracciabilità)
CREATE TABLE Voti (
    VotoID INT IDENTITY(1,1) PRIMARY KEY,
    StudenteID INT FOREIGN KEY REFERENCES Studenti(StudenteID),
    Materia VARCHAR(50) NOT NULL,
    Voto TINYINT NOT NULL CHECK (Voto BETWEEN 1 AND 10), -- Vincolo sul voto scolastico
    DataValutazione DATETIME2 DEFAULT GETDATE()
);
```

Modulo 4: Query Avanzate su ScuolaDB con Operatori e Filtri

Di seguito sono riportati alcuni scenari reali di estrazione dati basati sull'applicazione combinata dei tipi di dati e degli operatori studiati:

4.1 Query 1: Studenti maggiorenni di una specifica sezione con operatore logico

Vogliamo trovare gli studenti nati prima del 2008 iscritti alla sezione '5A':

```
SELECT S.Nome, S.Cognome, S.DataNascita
FROM Studenti S
INNER JOIN Classi C ON S.ClasseID = C.ClasseID
WHERE C.Sezione = '5A' AND S.DataNascita < '2008-01-01';
```

4.2 Query 2: Calcolo della media voti con filtro aggregato

Estraiamo la media dei voti per ogni materia, mostrando solo le materie che hanno una media superiore o uguale a 6 (voto di sufficienza), arrotondando il valore decimale:

```
SELECT Materia,
       CAST(AVG(CAST(Voto AS DECIMAL(4,2))) AS DECIMAL(4,2)) AS MediaVoti,
       COUNT(VotoID) AS NumeroVerificheSvolte
FROM Voti
GROUP BY Materia
HAVING AVG(CAST(Voto AS DECIMAL(4,2))) >= 6.0
ORDER BY MediaVoti DESC;
```

Nota tecnica sul CAST numerico

Poiché la colonna **Voto** è stata definita come **TINYINT** (intero), la funzione **AVG()** restituirebbe un numero intero troncando i decimali. Eseguire il **CAST** a un tipo **DECIMAL** permette di preservare i decimi e i centesimi nel calcolo della media scolastica.